

COMPTE RENDU : TP 1 JAVASCRIPT

Date : 9 Février 2026

Étudiant : Vongsanga Clément, Ben Oliel Hugo

Cours : Web Frontend et Backend

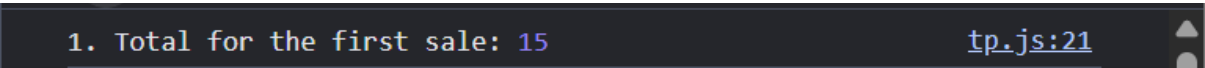
EXERCICE 1 : ANALYSE DE VENTES

1.1. Fonction classique `calculerTotal`

Cette fonction utilise la syntaxe traditionnelle pour multiplier la quantité par le prix unitaire d'un objet de vente.

```
// Tâche 1 : Fonction classique
function calculateTotal(sale) {
  return sale.quantity * sale.unitPrice;
}
```

Capture d'écran du résultat :



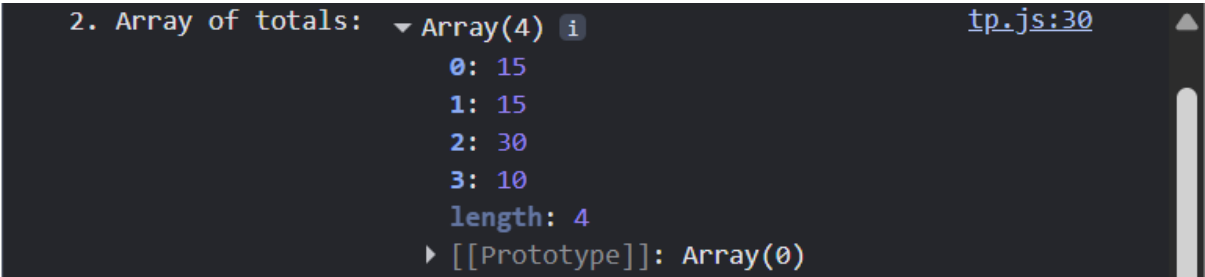
1. Total for the first sale: 15

tp.js:21

1.2. Traitement des données (map, filter, reduce)

Utilisation des méthodes modernes de tableaux pour transformer et filtrer les données.

```
// Tâche 2 : Création du tableau 'totaux' avec .map()
const totals = sales.map(calculateTotal);
```



2. Array of totals: ▼ Array(4) ⓘ

0: 15

1: 15

2: 30

3: 10

length: 4

▶ [[Prototype]]: Array(0)

tp.js:30

```
// Tâche 3 : Filtrage des ventes > 10 avec .filter()
const expensiveSales = sales.filter(sale => calculateTotal(sale) > 10);
```

3. Sales with total > 10: [tp.js:38](#)

```
▼ Array(3) i
  ▶ 0: {product: 'Stylo', quantity: 10, unitPrice: 1.5}
  ▶ 1: {product: 'Cahier', quantity: 5, unitPrice: 3}
  ▶ 2: {product: 'Livre', quantity: 2, unitPrice: 15}
    length: 3
  ▶ [[Prototype]]: Array(0)
```

```
// Tâche 4 : Calcul de la somme totale avec .reduce()
const totalSumExpensive = expensiveSales.reduce((acc, sale) => {
  return acc + calculateTotal(sale);
}, 0);
```

4. Sum of expensive sales: 60 [tp.js:49](#)

1.3. Variantes de fonctions fléchées (ES6)

Réécriture de la logique sous différentes formes de syntaxes fléchées.

```
// Tâche 5 :
// 1. Multi-lignes avec return explicite
// 2. Courte avec retour implicite
// 3. Retour d'un objet { produit, total }
const calculateTotalMultiLine = (sale) => {
  return sale.quantity * sale.unitPrice;
};

const calculateTotalShort = sale => sale.quantity *
sale.unitPrice;

const calculateTotalObject = sale => ({
  product: sale.product,
  total: sale.quantity * sale.unitPrice
});
```

5a. Multi-line: 15 [tp.js:72](#)

5b. Short: 15 [tp.js:73](#)

5c. Object return: ▶ Object [tp.js:74](#)

EXERCICE 2 : TRANSFORMATION DES VENTES

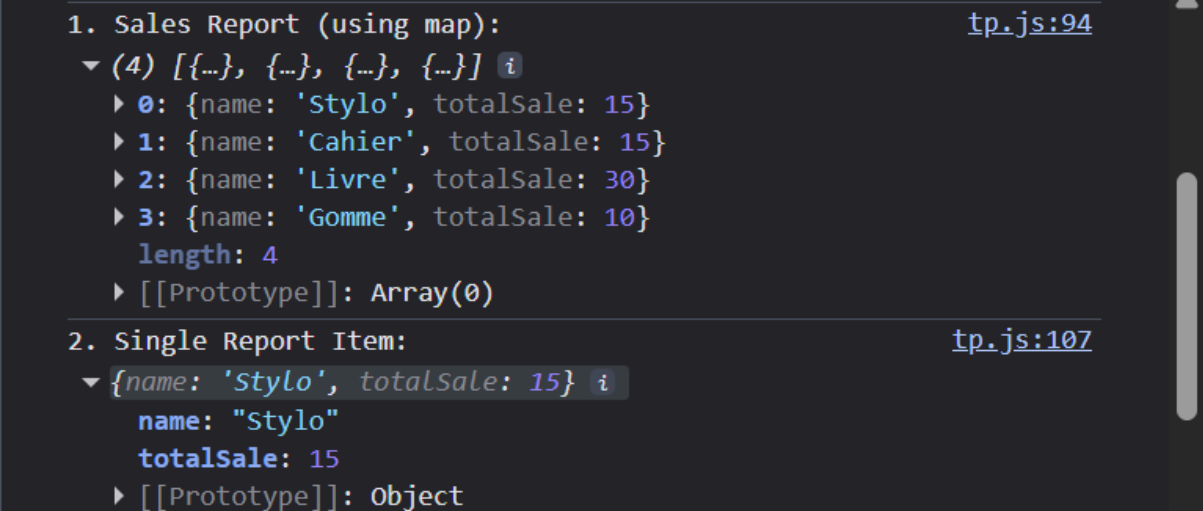
Objectif : Créer un rapport simplifié avec des noms de clés personnalisés.

2.1. Transformation du tableau

Utilisation de `map` pour créer un tableau contenant uniquement le nom du produit et le total de la vente.

```
// Tâche 1 & 2 : Transformation via map et fonction fléchée
const salesReport = sales.map(sale => {
  return {
    name: sale.product,
    totalSale: sale.quantity * sale.unitPrice
  };
});

const createReportItem = sale => ({
  name: sale.product,
  totalSale: sale.quantity * sale.unitPrice
});
```



```
1. Sales Report (using map): tp.js:94
  (4) [{...}, {...}, {...}, {...}] i
    0: {name: 'Stylo', totalSale: 15}
    1: {name: 'Cahier', totalSale: 15}
    2: {name: 'Livre', totalSale: 30}
    3: {name: 'Gomme', totalSale: 10}
    length: 4
    [[Prototype]]: Array(0)

2. Single Report Item: tp.js:107
  {name: 'Stylo', totalSale: 15} i
    name: "Stylo"
    totalSale: 15
    [[Prototype]]: Object
```

2.2. Comparaison des syntaxes

Démonstration de la même logique sous trois formes différentes.

```
// Tâche 3 :
// - Fonction classique
// - Arrow function avec return
// - Arrow function courte (retour implicite)
```

```

function createReportClassic(sale) {
  const total = sale.quantity * sale.unitPrice;
  return {
    name: sale.product,
    totalSale: total
  };
}

const createReportExplicit = (sale) => {
  const total = sale.quantity * sale.unitPrice;
  return {
    name: sale.product,
    totalSale: total
  };
};

const createReportImplicit = sale => ({
  name: sale.product,
  totalSale: sale.quantity * sale.unitPrice
});

```

```

3a. Classic:  ▾ {name: 'Cahier', totalSale: 15} ⓘ   tp.js:140
               name: "Cahier"
               totalSale: 15
               ▸ [[Prototype]]: Object

3b. Explicit Arrow:  tp.js:141
  ▾ {name: 'Cahier', totalSale: 15} ⓘ
    name: "Cahier"
    totalSale: 15
    ▸ [[Prototype]]: Object

3c. Implicit Arrow:  tp.js:142
  ▾ {name: 'Cahier', totalSale: 15} ⓘ
    name: "Cahier"
    totalSale: 15
    ▸ [[Prototype]]: Object

```

EXERCICE 3 : GESTION DES COMMANDES CLIENTS

Objectif : Traiter des structures de données imbriquées et filtrer par état de paiement.

3.1. Calcul et Filtrage des commandes

Calcul du montant total par commande et isolation des commandes payées.

```
// Tâche 1 : Fonction calculerTotalCommande
const calculateOrderTotal = order => order.products.reduce((acc,
product) => acc + product.price, 0);
```

1. Total for Alice's order: 16.5

[tp.js:162](#)

```
// Tâche 2 : Tableau 'commandesPayees' (filter)
const paidOrders = orders.filter(order => order.isPaid === true);
```

2. Paid orders:

[tp.js:171](#)

▼ (2) [{...}, {...}] i

▶ 0: {client: 'Alice', products: Array(2), isPaid: true}

▶ 1: {client: 'Charlie', products: Array(2), isPaid: true}

length: 2

▶ [[Prototype]]: Array(0)

3.2. Analyse financière des commandes payées

Extraction des montants et calcul de la recette globale.

```
// Tâche 3 : Tableau 'totauxPayees' (map)
const paidOrderTotals = paidOrders.map(calculateOrderTotal);
```

3. Totals of paid orders: ▼ (2) [16.5, 4.5] i

[tp.js:180](#)

0: 16.5

1: 4.5

length: 2

▶ [[Prototype]]: Array(0)

```
// Tâche 4 : Somme totale (reduce)
const totalRevenue = paidOrderTotals.reduce((acc, total) => acc +
total, 0);
```

4. Total Revenue (Paid): 21

[tp.js:189](#)

```
// Tâche 5 : Tableau 'rapportClients'
const clientReports = paidOrders.map(order => ({
  client: order.client,
  total: calculateOrderTotal(order)
})));
```

5. Client Reports:

[tp.js:201](#)

```
▼ (2) [{...}, {...}] i
  ▶ 0: {client: 'Alice', total: 16.5}
  ▶ 1: {client: 'Charlie', total: 4.5}
    length: 2
  ▶ [[Prototype]]: Array(0)
```

3.3. Bonus : Variantes de calcul

Réécriture de la logique de calcul de commande sous formes fléchées.

```
// Tâche 6 :
// - Courte avec retour implicite
// - Multi-lignes avec return
// - Retour d'objet { client, total }
const calcTotalShort = order => order.products.reduce((acc, p) =>
acc + p.price, 0);

const calcTotalMultiLine = (order) => {
  const total = order.products.reduce((acc, p) => acc + p.price,
0);
  return total;
};

const calcTotalObject = order => ({
  client: order.client,
  total: order.products.reduce((acc, p) => acc + p.price, 0)
});
```

6a. Short: 4.5

[tp.js:226](#)

6b. Multi-line: 4.5

[tp.js:227](#)

6c. Object Return:

```
▼ {client: 'Charlie', total: 4.5} i tp.js:228
  client: "Charlie"
  total: 4.5
  ▶ [[Prototype]]: Object
```